Omniversal Re-Presentation (CODA-OR): Re-Presentations Transport Rules

Author: Jeremy H. Carroll

Date: 2026-09-28

Version: v1.0.1 Pre-Print (1.0.1)

Status: first public preprint

License: Creative Commons Attribution 4.0 International

CODA Architecture & Layers

CODA stands for the Calculus of Obstruction, Descent, and Authorization. It is an exact, finite mathematical apparatus that enforces strict claim discipline by separating linear repairs, transport certificates, and finite descent from ungrounded authority claims. The system comprises eight operational layers governed by a single overarching authority architecture (GTLA).

Layer	Abbreviation	Layer / Component Name & Scope
1	**FOC**	Finite Obstruction Calculus
2	**MUR**	Meta & Universal Re-Presentation (what presentations exist)
3	**CS**	Continuum Shadow (what continuum presentation may descend to a finite calculus system)
4	**CS_Q**	Continuum Shadow to Continuum Bridge (how continuum presentations descend)
5	**Q**	The Continuum (continuum presentations via completion and enclosure)
6	**A**	Anomalon (global residuals not removed and typed instead)
7	**OR**	Omniversal Re-Presentation (what presentations can transport)
8	**FDA**	Finite Descent Authority (what presentation may descend to evaluation and authorization)
9	**GTLA**	Layered Authority Architecture for CODA

Abstract

Two overlapping pairs glue to a unique triple exactly when they agree on the shared slot. A comparison envelope organizes such gluings, limits, and completions. Membership in that envelope is not standing. An empty support sieve — no incoming arrow that both carries source standing and a certificate — is a hard refusal. A limit candidate missing a local terminalizer has no limit standing. Diagram squares are labelled in four ways by leftover and closure: strict commute, exact repair, closed leftover, or open leftover. Exact repairs are closed because $d_1 d_0 = 0$. A global "omniversal" object is not assumed.

Introduction

It is easy to treat a completed diagram as a certified object. You form a limit, you fill a square, you adjoin a comparison node, and you talk as if the new node inherited every receipt the old nodes had. This paper separates the diagram from the receipts.

A picture. Two pairs (a, b) and (b', c) want to become a triple (a, b, c). They glue if and only if $b = b'$. If one pair ends in `false` and the other starts in `true`, there is no triple. That is gluing, not standing.

A second picture. Four squares in a one-dimensional cochain $C^0 = C^1 = C^2 = \mathbb{Q}$:

1. raw defect 0 — strict commute;
2. raw defect 3, repair map $d_0 = 1$, next map $d_1 = 0$ — exact repair, leftover 0;
3. raw defect 3, $d_0 = 0$, $d_1 = 0$ — closed leftover 3;
4. raw defect 3, $d_0 = 0$, $d_1 = 1$ — open leftover 3.

The theory is refuted by any of: a unique glue of incompatible overlaps; an envelope member treated as standing with an empty support sieve; a limit candidate with no local terminalizer treated as standing; an exact repair that is not closed; an ℓ^2 leftover substituted for the ℓ^1 quotient on a classified square.

This paper does not prove a bicategory of all comparison systems, and it does not construct a global terminal object of all regimes.

Notation

- Pairs $(a, b) \in A \times B$ and $(b', c) \in B \times C$ are *compatible* when $b = b'$.
- An *enriched environment* is finite dimensions and matrices $d_0 \colon C_0 \to C_1$, $d_1 \colon C_1 \to C_2$ with $d_1 d_0 = 0$ exactly.
- A *raw defect* δ is a column in C_1.
- $\Phi_1([\delta])$ is the cheapest ℓ^1 leftover after adding $d_0\alpha$.
- A *support arrow* into X names a source, whether that source has standing, and an optional certificate id.
- A *support sieve* of X is the list of such arrows. It is empty when no arrow has both source standing and a certificate.
- A *comparison object* has an id, a weight class, a support sieve, and optional descent, verifier, and terminalizer names.

1. Gluing on an overlap

Definition OR-D1 — compatibility

Pairs left $\in A \times B$ and right $\in B \times C$ are compatible when left.2 = right.1.

Theorem OR-T1 — compatible pairs glue uniquely

If the pairs are compatible, there is a unique triple $t \in A \times B \times C$ whose first two slots recover the left pair and whose last two slots recover the right pair.

Proof. Take $t = (\text{left.1}, \text{left.2}, \text{right.2})$. The overlap equality supplies the second projection. Any other triple with the same projections equals t componentwise. This is Lean `pair_gluing`. □

Theorem OR-T2 — a glue requires overlap

If some triple recovers both pairs, the pairs are compatible.

Proof. The shared B-slot is both left.2 and right.1. This is Lean `gluing_requires_overlap`. □

Theorem OR-T3 — incompatible countermodel

There is no triple of type $\mathbf{1} \times \mathsf{Bool} \times \mathbf{1}$ recovering $(*, \mathsf{false})$ and $(\mathsf{true}, *)$.

Proof. Apply OR-T2. This is Lean `incompatible_countermodel`. □

2. Four labels for a square

Definition OR-D2 — square status

Given an enriched environment and a defect $\delta \in C_1$:

1. *Strict commute* if $\delta = 0$;
2. *Exact repair* if $\delta \neq 0$ and $\delta = d_0\alpha$ for some α;
3. *Closed leftover* if δ is not exact and $d_1\delta = 0$;
4. *Open leftover* if $d_1\delta \neq 0$.

The calculator uses the ℓ^1 leftover $\Phi_1([\delta]) = 0$ as the numerical stand-in for exactness when d_0 is presented as a matrix.

Theorem OR-T4 — exact repairs are closed

If $\delta = d_0\alpha$, then $d_1\delta = 0$.

Proof. $d_1 d_0 = 0$. This is Lean `exact_repair_is_closed`. □

So an open leftover cannot be exact. Strict commute is closed because $d_1(0) = 0$.

The four-row worked example of the introduction is the companion function `reproduce_section_6a_worked_example`. Substituting ℓ^2 for ℓ^1 on $A = (1, 1, 1)^T$, $x = (0, 0, 10)$ is a killed mutant.

3. Empty support is a refusal

Definition OR-D3 — standing of a comparison object

In the Lean core, standing of X requires a nonempty support sieve, a local terminalizer flag, and a descent-triplet flag. In the calculator, `evaluate_comparison_authority` refuses an empty sieve (`R_empty_support`) before it looks at descent, verifier, and terminalizer names, and refuses a missing name triple (`R_no_terminalizer`) next.

Theorem OR-T5 — an empty sieve implies no standing

If the support sieve of X is empty, X has no standing.

Proof. Standing stores a proof that the sieve is nonempty. This is Lean `empty_support_sieve_implies_zero_authority`. □

An arrow whose source lacks standing, or that lacks a certificate, is not support. Those are Lean `uncredentialed_source_not_auth_supporting` and `uncertified_arrow_not_auth_supporting`.

Theorem OR-T6 — envelope membership is not standing

There exists a comparison object — empty sieve, no local terminalizer, no descent triplet — that has no standing. In particular, standing is not true of every comparison object.

Proof. The uncredentialed witness. Absence of a terminalizer blocks standing. This is Lean `completion_reflector_does_not_create_authority` and `completion_membership_not_implies_authority`. □

Putting X into the envelope is adjoining a node. It is not a receipt.

4. Limits still need a terminalizer

Definition OR-D4 — limit candidate

A limit candidate has a list of cone legs and a declared-terminalizer flag. A leg is valid when its component has standing and its projection has a certificate. Limit standing is: every leg valid, and the terminalizer flag true.

Theorem OR-T7 — missing terminalizer, no limit standing

If the declared-terminalizer flag is false, the candidate has no limit standing.

Proof. Limit standing stores that flag. This is Lean `limit_missing_terminalizer_lacks_authority`. □

An invalid leg likewise blocks limit standing. This is Lean `limit_invalid_leg_lacks_authority`.

Theorem OR-T8 — a global omniversal object is not assumed

The proposition "a global omniversal comparison object is assumed" is definitionally false.

Proof. It is defined as `False`. This is Lean `omniversal_not_assumed`. □

That is a refusal to assume, not a proof that no such object exists.

4.1 Finite Authority & Non-Terminal Computation (Paper 028)

This section synthesizes the non-terminal computation halting discipline from Paper 028 (*Finite Authority and Non-Terminal Computation*):

Reasoning authority is not generated; it is terminalized.

Universal symbol manipulation is not universal authority.

Scope fence OR-T9 — Reasoning Authority Non-Generation Fence

A proposal system $\mathcal{P}$ (e.g. an LLM, heuristic solver, or unconstrained hypothesis search) generating reasoning trajectories acquires no authority by virtue of generation depth, trace length, or heuristic confidence:

$$\forall \tau \in \mathrm{Traces}(\mathcal{P}), \quad \tau \not\Rightarrow \mathsf{Auth}(\tau).$$

Authority exists solely as an evaluated terminal outcome produced by an authorized terminalizer over a finite, declared boundary.

Policy OR-T10 — Non-Terminal Computation Halting Discipline

Retyped 2026-09-23. OR-T10 is a declared policy, not a theorem: it constrains what a pipeline must do and has no proof. Its former evidence, `apps/coda/tests/test_high_dim_stress.py` as it then stood, never exhausted a budget, and that app checked its budget only between layers. The "> 3.0 s" figure belonged to one app configuration, not to the claim; it is **WITHDRAWN 2026-09-23**. `apps/coda` now ticks its budget on every solver pivot. Its tests `test_budget_exhaustion_inside_layer_one_refuses`

and `test_budget_exhaustion_by_step_limit_refuses` check, on finite instances, that exhaustion refuses. They illustrate the policy; they do not prove it.

Any computation that fails to terminate at an admissible terminalizer within declared resource budgets (a time limit, a step limit) or whose candidate support sieve is empty ($\mathrm{Sieve}(E) = \emptyset$) must fail closed. *including runtime bounds ($> 3.0s$ on high-dimensional instances)*

$$\mathrm{BudgetExhausted}(p) \vee (\mathrm{Sieve}(E) = \emptyset) \implies \Pi^{\sharp}(p) = \mathsf{inr}(\mathrm{Refusal}(\mathbb{F}_{\mathrm{term}})).$$

The pipeline strictly refuses heuristic guessing, stochastic fallback, or majority-vote promotion.

5. Computational companion

The production module `code/coda_or_core.py` classifies squares with an exact rational ℓ^1 leftover, refuses $d_1 d_0 \neq 0$, refuses empty support sieves, and refuses missing descent/verifier/terminalizer names. Floats and booleans in scalar position raise `TypeError`. Protected skeletons raise on unledgered mutation.

Lean files in `lean4/`:

- records for envelopes, sieves, and complexes;
- non-creation and support-sieve kernels — Theorems OR-T5–T6;
- square classification — Theorem OR-T4 and the four labels;
- limit standing — Theorems OR-T7–T8;
- `Algebra.lean` — wraps reconstruction Theorems OR-T1–T3;
- `Core.lean` — re-exports the files above.

There are no `sorry` stubs in those files. A bicategory of all comparison systems is not claimed.

Adversarial tests include: empty sieve with names filled in; closed leftover labelled as exact; ℓ^1 versus ℓ^2 leftover; $d_1 d_0 \neq 0$ refused at environment construction.

6. Limitations

Gluing is uniqueness of a triple of types, not a 2-categorical colimit. The four labels are the zero / exact / closed / open split of a finite cochain defect. The leftover solver, when used, measures leftover. Completing a diagram does not mint a later policy.

Formal target-binding correspondence

Lean `HasAuthority X` now includes `X.sieveTargetBound`: the sieve targets `X.object_id` and every arrow targets that sieve. The old three-flag predicate admitted an object `X`, sieve `Z` and supporting arrow into `Y`. The declarations `old_flags_admit_mismatched_targets` and `mismatched_targets_have_no_authority` record that counterexample and its rejection under the corrected predicate. `bound_target_example_has_authority` supplies a valid finite example.

On records with valid nonblank names and declared weight class, Python with `require_all_sources=False` checks the same support, local flags, and target agreement. Python additionally validates its concrete string fields and returns diagnostic payloads. Its default `require_all_sources=True` requires every arrow to support standing. Lean's separate `LimitCandidate` record has its own cone-leg predicate; this is not a full formal equivalence proof of the Python API.

References

1. S. Mac Lane, *Categories for the Working Mathematician*, 2nd ed., Springer,

1998. https://doi.org/10.1007/978-1-4757-4721-8
2. P. Gabriel and M. Zisman, *Calculus of Fractions and Homotopy Theory*, Springer, 1967.
3. A. Grothendieck, "Technique de descente et théorèmes d'existence en géométrie algébrique," Séminaire Bourbaki, 1959.
4. R. T. Rockafellar, *Convex Analysis*, Princeton University Press, 1970.